

3.7.3 Segmentation with Paging: The Intel x86

Up until the x86-64, the virtual memory system of the x86 resembled that of MULTICS in many ways, including the presence of both segmentation and paging. Whereas MULTICS had 256K independent segments, each up to 64K 36-bit words, the x86 has 16K independent segments, each holding up to 1 billion 32-bit words. Although there are fewer segments, the larger segment size is far more important, as few programs need more than 1000 segments, but many programs need large segments. As of x86-64, segmentation is considered obsolete and is no longer supported, except in legacy mode. Although some vestiges of the old segmentation mechanisms are still available in x86-64's native mode, mostly for compatibility, they no longer serve the same role and no longer offer true segmentation. The x86-32, however, still comes equipped with the whole shebang.

So why did Intel kill what was a variant of the perfectly good MULTICS memory model that it supported for close to three decades? Probably the main reason is that neither UNIX nor Windows ever used it, even though it was quite efficient because it eliminated system calls, turning them into lightning-fast procedure calls to the relevant address within a protected operating system segment. None of the developers of any UNIX or Windows system wanted to change their memory model to something that was x86 specific because it would surely break portability to other platforms. Since the software was not using the feature, Intel got tired of wasting chip area to support it and removed it from the 64-bit CPUs.

All in all, one has to give credit to the x86 designers. Given the conflicting goals of implementing pure paging, pure segmentation, and paged segments, while at the same time being compatible with the 286, and doing all of this efficiently, the resulting design is surprisingly simple and clean.

3.8 RESEARCH ON MEMORY MANAGEMENT

Memory management is an active research area and every year brings a new harvest of publications to improve a system's security, performance, or both. Also, while traditional memory management topics, especially paging algorithms for uniprocessor CPUs, has largely died off, researchers now look at new types of storage, or at incorporating memory on remote machines (Ruan et al., 2020). Also, some people never say die and even the good old fork system call has been given a do-over. Observing that the performance of fork has become a bottleneck for memory intensive applications, as all the page tables must be copied first, even if the data and code pages themselves are shared copy-on-write, as a logical next step, researchers decided to also share the page tables copy-on-write (Zhao, 2021).

Memory management in datacenters and clouds is complicated. For instance, a big problem arises when a virtual machine is happily humming along when all of a

sudden the hypervisor needs an update. While you may migrate the virtual machines to another node, this turns out to be inefficient and in-place updates are often possible, even without a reboot, by preserving the memory pages of the virtual machine (Russeinovich, 2021). Another issue that makes memory management in these environments different from traditional settings is that in data centers many applications run on a complex software stack where each layer does some memory management and the applications are able to adapt their performance to the available memory, rendering the working set model ineffective. Performance can be improved by inserting policies and mechanisms in each layer to coordinate memory management (Lion et al., 2021).

Integrating new forms of memory and storage into the regular memory hierarchy is not easy and the adoption of persistent memory in existing systems has been a bumpy ride (Neal et al., 2020). Much research tries to make the integration more seamless. For instance, researchers have designed techniques for conversion of regular DRAM addresses to persistent memory addresses (Lee et al., 2019). Others have used persistent memory to convert existing distributed in-memory storage systems into persistent, crash-consistent versions with low overhead and minimal code changes (Zhang et al., 2020).

Many aspects of memory management have become a battleground for security researchers. For instance, reducing the memory footprint of a system by means of memory deduplication turns out to be a highly security sensitive operation. Who knew? For instance, attackers may detect that a page has been deduplicated and thus learn what another process has in its address space (Bosman and Bos, 2016). To eliminate the threat, deduplication must be designed such that one can no longer distinguish between deduplicated and non-deduplicated pages (Oliverio, et al., 2017).

Many attacks on the operating system depend on getting memory lined up in the right way. For instance, attackers may be able to corrupt some important value (e.g., the return address from a procedure call), but only if the corresponding object is at a specific location. Such heap lay-out feng shui is complicated to perform from a user process and researchers have looked for ways of automating this process (Chen and Xing, 2019).

Finally, there has been substantial work on operating systems due to Meltdown and Spectre vulnerabilities in popular CPUs (see also Chapter 9). In particular, it has led to radical and expensive changes in Linux on many processors. Where the Linux kernel was originally mapped into the address space of every process as a measure to speed up system calls (by obviating the need to change page tables for a system call), Meltdown required strict page table isolation. As it made the context switch much more expensive, the Linux developers were furious with Intel. The names initially proposed for the expensive fix were “User Address Space Separation” and “Forcefully Unmap Complete Kernel With Interrupt Trampolines,” but eventually they settled on kernel page table isolation (kpti), as the acronym is considerably less offensive.

3.9 SUMMARY

In this chapter, we have examined memory management. We saw that the simplest systems do not swap or page at all. Once a program is loaded into memory, it remains there in place until it finishes. Some operating systems allow only one process at a time in memory, while others support multiprogramming. This model is still common in small, embedded real-time systems.

The next step up is swapping. When swapping is used, the system can handle more processes than it has room for in memory. Processes for which there is no room are swapped out to the disk or SSD. Free space in memory and on non-volatile storage can be kept track of with a bitmap or a hole list.

Modern computers often have some form of virtual memory. In the simplest form, each process' address space is divided up into uniform-sized blocks called pages, which can be placed into any available page frame in memory. There are many page replacement algorithms; two of the better algorithms are aging and WSClock.

To make paging systems work well, choosing an algorithm is not enough; attention to such issues as determining the working set, memory allocation policy, and page size is required.

Segmentation helps in handling data structures that can change size during execution and simplifies linking and sharing. It also facilitates providing different protection for different segments. Sometimes segmentation and paging are combined to provide a two-dimensional virtual memory. The MULTICS system and the 32-bit Intel x86 support segmentation and paging. Still, it is clear that few operating system developers care deeply about segmentation (because they are married to a different memory model).

PROBLEMS

1. In Fig. 3-3 the base and limit registers contain the same value, 16,384. Is this just an accident, or are they always the same? If it is not an accident, why are they the same in this example?
2. In this problem, you are to compare the storage needed to keep track of free memory using a bitmap versus using a linked list. The 8-GB memory is allocated in units of n bytes. For the linked list, assume that memory consists of an alternating sequence of segments and holes, each 1 MB. Also assume that each node in the linked list needs a 32-bit memory address, a 16-bit length, and a 16-bit next-node field. How many bytes of storage is required for each method? Which one is better?